

R308 - TP2

Partie 1 : Classe Date & Exceptions (60 minutes)

Exercice 1 — Classe Date (30 min)

Objectifs : validation, invariants, ordre, parsing/formatting.

Énoncé :

Créez `datex.py`. Implémentez une classe `Date` (calendrier grégorien) sans utiliser `datetime.date` pour la validation (vous pouvez l'utiliser pour les tests si besoin).

Contraintes fonctionnelles :

- Constructeur : `__init__ (self, year:int, month:int, day:int)`
 - Valider : $\text{year} \in [1, 9999]$, $\text{month} \in [1, 12]$, day valide selon le mois et les années bissextiles.
 - En cas d'erreur : lever `InvalidDateError`.
- Méthodes & propriétés :
 - `is_leap_year(self) -> bool`
 - `__repr__` et `__str__` (formats `Date(2025,09,22)` et `2025-09-22`)
 - Comparaisons : `__eq__`, `__lt__` (et déduire les autres via `functools.total_ordering`).
 - `to_iso(self) -> str (YYYY-MM-DD)`
 - `@classmethod from_iso(cls, s:str) -> "Date"` : parser `YYYY-MM-DD` ou lever `DateParseError`.
- Méthodes utilitaires :
 - `day_of_year(self) -> int`
 - `add_days(self, n:int) -> "Date"` (retourne un **nouvel** objet, ne modifie pas `self`).

Exceptions à définir :

```
# errors.py
class DateError(Exception): ...
class InvalidDateError(DateError): ...
class DateParseError(DateError): ...
```

Code de départ (datex.py) :

```
from __future__ import annotations
from functools import total_ordering
from typing import Tuple
from errors import InvalidDateError, DateParseError

@total_ordering
class Date:
    """Date grégorienne minimale, immuable en pratique (méthodes renvoient des copies)."""

    __slots__ = ("_y", "_m", "_d")

    def __init__(self, year: int, month: int, day: int):
        # TODO: valider et affecter _y, _m, _d ; lever InvalidDateError si invalide
        pass

    @property
    def year(self) -> int: return self._y
    @property
    def month(self) -> int: return self._m
    @property
    def day(self) -> int: return self._d

    def __repr__(self) -> str:
        # TODO: format non ambigu
        return ""

    def __str__(self) -> str:
        # TODO: format lisible YYYY-MM-DD
        return ""
```

```

def to_iso(self) -> str:
    # TODO
    return ""

@classmethod
def from_iso(cls, s: str) -> "Date":
    # TODO: parser, lever DateParseError si invalide
    return cls(1970, 1, 1)

def is_leap_year(self) -> bool:
    # TODO
    return False

def _days_in_month(self, y: int, m: int) -> int:
    # TODO
    return 30

def day_of_year(self) -> int:
    # TODO
    return 0

def add_days(self, n: int) -> "Date":
    # TODO: retourner une nouvelle date, gérer dépassements
    # mois/année
    return self

def __eq__(self, other: object) -> bool:
    if not isinstance(other, Date):
        return NotImplemented
    # TODO
    return False

def __lt__(self, other: "Date") -> bool:
    # TODO: ordre lexicographique (y,m,d)
    return False

# --- Tests rapides ---

```

```

if __name__ == "__main__":
    d = Date(2024, 2, 29)
    assert d.is_leap_year() is True
    assert str(d) == "2024-02-29"
    assert Date.from_iso("2025-09-22").to_iso() == "2025-09-22"

    try:
        Date(2023, 2, 29)
        assert False, "Devrait lever InvalidDateError"
    except InvalidDateError:
        pass

    try:
        Date.from_iso("2025-13-01")
        assert False, "Devrait lever DateParseError"
    except DateParseError:
        pass

    # add_days et ordre
    d1 = Date(2025, 12, 31).add_days(1)
    assert str(d1) == "2026-01-01"
    assert Date(2025, 1, 1) < Date(2025, 1, 2)

    print("Tests Ex1 OK")

```

datex.py

errors.py

Exercice 2 — DateRange & recouvrement (30 min)

Objectifs : types composés, invariants, égalité, méthodes métier + erreurs.

Énoncé :

Créez `daterange.py` avec une classe `DateRange(start:Date, end:Date)` **include** (les bornes inclusives).

- Lever `InvalidDateError` si `end < start`.
- Méthodes :
 - `duration(self) -> int` (nb de jours, ex. `[2025-09-01..2025-09-01] → 1`)
 - `contains(self, d:Date) -> bool`
 - `overlaps(self, other:"DateRange") -> bool`
 - `intersection(self, other:"DateRange") -> "DateRange | None"`

Code de départ (daterange.py) :

```
from __future__ import annotations
from datex import Date
from errors import InvalidDateError

class DateRange:
    __slots__ = ("start", "end")

    def __init__(self, start: Date, end: Date):
        # TODO: valider start <= end
        pass

    def duration(self) -> int:
        # TODO
        return 0

    def contains(self, d: Date) -> bool:
        # TODO
        return False

    def overlaps(self, other: "DateRange") -> bool:
        # TODO
        return False

    def intersection(self, other: "DateRange") -> "DateRang
```

```

e | None":
    # TODO
    return None

# --- Tests rapides ---
if __name__ == "__main__":
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
    b = DateRange(Date(2025, 9, 5), Date(2025, 9, 15))
    c = a.intersection(b)
    assert c is not None and c.start.to_iso() == "2025-09-05" and c.end.to_iso() == "2025-09-10"
    assert a.duration() == 10
    assert a.contains(Date(2025, 9, 1)) and a.contains(Date(2025, 9, 10))
    print("Tests Ex2 OK")

```

daterange.py

Partie 2 : Sérialisation Texte vs Binaire (45 minutes)

Exercice 3 — JSON/CSV (texte) & Pickle (binaire) (45 min)

Objectifs : sérialiser une collection d'objets métier, comparer les formats.

Énoncé :

Créez `serialization.py` avec des fonctions pour sérialiser une liste de Date :

- **Texte JSON (JSON Lines) :** `dump_dates_jsonl(dates, path)`, `load_dates_jsonl(path)` Une ligne par objet : `{"year":2025,"month":9,"day":22}`
- **CSV :** `dump_dates_csv(dates, path)`, `load_dates_csv(path)`
- **Binaire (pickle) :** `dump_dates_pickle(dates, path)`, `load_dates_pickle(path)`

Contraintes :

- Validation lors du chargement : lever `DateParseError` si enregistrement invalide.

- Gérer proprement I/O : `with open(..., "w", encoding="utf-8")`.
- Fournir un mini-bench : sauvegarder 10000 dates, comparer tailles de fichiers.

Code de départ (`serialization.py`) :

```
from __future__ import annotations
import json, csv, pickle, os
from typing import Iterable, List
from datex import Date
from errors import DateParseError

def dump_dates_jsonl(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass

def load_dates_jsonl(path: str) -> List[Date]:
    # TODO
    return []

def dump_dates_csv(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass

def load_dates_csv(path: str) -> List[Date]:
    # TODO
    return []

def dump_dates_pickle(dates: Iterable[Date], path: str) -> None:
    # TODO
    pass

def load_dates_pickle(path: str) -> List[Date]:
    # TODO
    return []
```

```
# --- Tests/bench ---
if __name__ == "__main__":
    # Générer des dates
    base = Date(2025, 1, 1)
    dates = [base.add_days(i) for i in range(10000)]

    dump_dates_jsonl(dates, "dates.jsonl")
    dump_dates_csv(dates, "dates.csv")
    dump_dates_pickle(dates, "dates.pkl")

    lj = load_dates_jsonl("dates.jsonl")
    lc = load_dates_csv("dates.csv")
    lp = load_dates_pickle("dates.pkl")

    assert lj[:3][0].to_iso() == "2025-01-01"
    assert lc[-1].to_iso() == "2052-05-18" # 10000e jour
    assert lp[1234].to_iso() == dates[1234].to_iso()

    sizes = {p: os.path.getsize(p) for p in ["dates.jsonl", "dates.csv", "dates.pkl"]}
    print("Tailles (octets):", sizes)
    # Discussion attendue en compte-rendu: lisibilité vs compacité vs portabilité
```

[serialization.py](#)

Partie 3 : Gestion des erreurs / exceptions (30 minutes)

Exercice 4 — API robuste & journalisation légère (30 min)

Objectifs : maîtriser `try / except / else / finally`, exceptions métiers, messages utiles.

Énoncé :

Créez `repo.py` : un dépôt simple pour stocker des `DateRange` en `JSONL`.

- Exceptions :

```
class RepositoryError(Exception): ...
class RepositoryIOError(RepositoryError): ...
class RepositoryDataError(RepositoryError): ...
```

- Classe `DateRangeRepository(path:str)` avec :
 - `save(self, ranges:list[DateRange]) -> None`
 - `load(self) -> list[DateRange]`
- Règles :
 - I/O : capturer `OSError` → `RepositoryIOError` (enrichir le message avec le chemin).
 - Données invalides (bornes inversées, champs manquants) → `RepositoryDataError`.
 - Utiliser `try / except / else / finally` pour séparer logique et nettoyage.
 - (Optionnel) journalisation minimaliste avec `print("[INFO] ...")` pour l'exercice.

Code de départ (repo.py) :

```
from __future__ import annotations
import json
from typing import List, Dict, Any
from datex import Date
from daterange import DateRange
from errors import InvalidDateError
from serialization import DateParseError # ou depuis error
s si déplacé

class RepositoryError(Exception): ...
class RepositoryIOError(RepositoryError): ...
class RepositoryDataError(RepositoryError): ...

class DateRangeRepository:
    def __init__(self, path: str):
        self._path = path
```

```

def save(self, ranges: List[DateRange]) -> None:
    # TODO: sérialiser en JSONL avec schéma {"start": "Y
YYY-MM-DD", "end": "YYYY-MM-DD"}
    pass

def load(self) -> List[DateRange]:
    # TODO: I/O + validation + mapping vers objets
    return []

# --- Tests rapides ---
if __name__ == "__main__":
    repo = DateRangeRepository("ranges.jsonl")
    a = DateRange(Date(2025, 9, 1), Date(2025, 9, 10))
    b = DateRange(Date(2025, 9, 20), Date(2025, 9, 21))
    repo.save([a, b])
    loaded = repo.load()
    assert len(loaded) == 2 and loaded[0].start.to_iso() ==
"2025-09-01"
    print("Tests Ex4 OK")

```

repo.py

Partie 4 : Mini-Projet de Synthèse (45 minutes)

Exercice 5 — Gestion d'« Événements » avec dépôt pluggable

Objectifs : intégrer `Date`, `DateRange`, sérialisation, exceptions, et interface dépôt.

Contexte :

On veut gérer des événements avec une date unique ou une période. Chaque événement a : `title:str`, `period:DateRange`, `tags:list[str]`.

Énoncé :

Créez `events.py` et `event_repo.py`.

1. Event :

- `__init__(self, title:str, period:DateRange, tags:list[str] | None=None)`
- `__repr__`, `__str__` → ex: `Event('Conf IAG', [2025-10-06..2025-10-06], tags=['univ','iag'])`
- `overlaps(self, other:"Event") -> bool` (via `DateRange.overlaps`)

2. Dépôts :

- Interface minimale `BaseEventRepository` (classe abstraite simple) avec `save(events)`, `load()`.
- Implémentations :
 - `EventJsonlRepository(path)` : texte `JSONL` (lisible).
 - `EventBinaryRepository(path)` : binaire (pickle).

3. Script `main.py` :

- Créer 5–10 événements (mélanger dates uniques et périodes).
- Sauvegarder via `JSONL` puis `Pickle`.
- Recharger et vérifier :
 - même nombre,
 - détection de conflits (deux événements qui se chevauchent).
- Afficher la taille des fichiers et un court commentaire (« texte plus portable/lisible, binaire plus compact/rapide mais moins sûr et moins interopérable »).

Squelettes :

```
# events.py
from __future__ import annotations
from typing import List
from daterange import DateRange

class Event:
    __slots__ = ("title", "period", "tags")
    def __init__(self, title: str, period: DateRange, tags:
List[str] | None = None):
        # TODO: valider title non vide, copier tags or []
        pass
    def overlaps(self, other: "Event") -> bool:
```

```

        # TODO
        return False
    def __repr__(self) -> str:
        # TODO
        return ""
    def __str__(self) -> str:
        # TODO
        return ""

# event_repo.py
from __future__ import annotations
import json, pickle, os
from typing import List, Dict, Any
from events import Event
from datex import Date
from daterange import DateRange
from repo import RepositoryError, RepositoryIOError, RepositoryDataError

class BaseEventRepository:
    def save(self, events: List[Event]) -> None: raise NotImplementedError
    def load(self) -> List[Event]: raise NotImplementedError

class EventJsonlRepository(BaseEventRepository):
    def __init__(self, path: str): self._path = path
    def save(self, events: List[Event]) -> None:
        # TODO: {"title":..., "start":"YYYY-MM-DD", "end":"YYYY-MM-DD", "tags":[...]}
        pass
    def load(self) -> List[Event]:
        # TODO: validations + exceptions
        return []

class EventBinaryRepository(BaseEventRepository):
    def __init__(self, path: str): self._path = path
    def save(self, events: List[Event]) -> None:

```

```

        # TODO: pickle.dump
        pass
    def load(self) -> List[Event]:
        # TODO: pickle.load + validations minimales
        return []

```

```

# main.py (mini-projet)
from __future__ import annotations
from events import Event
from daterange import DateRange
from datex import Date
from event_repo import EventJsonlRepository, EventBinaryRepository

if __name__ == "__main__":
    evs = [
        Event("Conf IAG", DateRange(Date(2025,10,6), Date(2025,10,6)), ["univ", "iag"]),
        Event("Soutenance", DateRange(Date(2025,11,3), Date(2025,11,3)), ["phd"]),
        Event("Vacances Toussaint", DateRange(Date(2025,10,25), Date(2025,11,2)), ["feries"]),
        Event("Jury", DateRange(Date(2025,11,2), Date(2025,11,2

```

events.py

event_repo.py

main.py